

Credit Risk Prediction for SBA Loans Using XGBoost and Explainable AI

Harshil Shinde
Computational Data Science
College of Engineering
Pennsylvania State University
University Park, PA, USA
hks5537@psu.edu

Samarth Patel
Computational Data Science
College of Engineering
Pennsylvania State University
University Park, PA, USA
shp5246@psu.edu

Abstract—Small businesses rely heavily on access to credit, yet lenders face the challenge of accurately assessing default risk for new and existing borrowers. In this work, we build a credit risk prediction model for U.S. Small Business Administration (SBA) 7(a) loans using the public FOIA 7(a) loan-level dataset for fiscal year 2020 onward. Inspired by a recent study that compared several machine-learning algorithms for retail credit scoring, we develop an end-to-end pipeline that (i) constructs a performance window based on months since loan origination, (ii) performs exploratory data analysis (EDA) on key loan attributes, and (iii) trains a tuned XGBoost classifier within a scikit-learn pipeline. To handle strong class imbalance between non-default and default cases, we incorporate class weighting through the `scale_pos_weight` parameter and use stratified train-test splitting. On a held-out test set, our tuned XGBoost model achieves accuracy of about 0.80, precision of about 0.21, recall of about 0.60 for the minority (default) class, an F1 score of about 0.31, and ROC-AUC of about 0.79.¹ We further apply SHAP values to interpret the model and identify the most important drivers of default risk, such as guaranteed approval amount, total approval amount, and jobs supported. The results suggest that gradient-boosted decision trees combined with explainable AI tools provide a practical and transparent approach for SBA loan risk monitoring.

Index Terms—credit risk, SBA 7(a) loans, machine learning, XGBoost, class imbalance, SHAP, explainable AI

I. INTRODUCTION

The U.S. Small Business Administration (SBA) 7(a) loan program is one of the primary sources of government-backed financing for small businesses. By guaranteeing a portion of the loan principal, the SBA encourages banks and other lenders to extend credit to borrowers that might otherwise struggle to obtain funding. However, these guarantees also create an incentive for careful risk management: misjudging default risk can result in substantial losses to both lenders and taxpayers.

Recent advances in machine learning (ML) have enabled more flexible and accurate credit scoring systems, especially when rich transactional or loan-level data are available. A recent study published in the journal *Risks* developed and compared several ML models—including logistic regression, neural networks, random forest, XGBoost, LightGBM, and

AdaBoost—for predicting retail credit default, with extensive analysis of confusion matrices and performance metrics such as ROC-AUC and Matthews correlation coefficient (MCC). [1] That work motivates the use of tree-based boosting algorithms and careful handling of class imbalance in credit risk modeling.

In this paper, we adapt these ideas to the SBA 7(a) context using the SBA FOIA 7(a) loan dataset for fiscal year (FY) 2020 through the most recent available period. We focus on three simple but practical questions:

- What does the SBA 7(a) portfolio’s bad-rate pattern look like over time when we measure by months since loan approval?
- How do basic loan features (approval amount, guaranteed amount, SBA share, jobs supported, business type, and delivery method) differ between good and bad loans?
- If we train a tuned XGBoost model on these engineered features, how well can it separate good and bad loans on a held-out test set, and which inputs does it rely on most according to SHAP explanations?

Our contributions are threefold. First, we construct a performance-window view of the SBA 7(a) portfolio, examining how bad rates change as loans season. Second, we build a reproducible EDA and modeling pipeline in Python that preprocesses FOIA data, handles class imbalance using class weighting, and trains a tuned XGBoost model. Third, we apply SHAP-based explainability to understand which features most strongly influence the model’s predictions and how those patterns align with domain intuition.

II. RELATED WORK

Classical credit scoring approaches rely on logistic regression models applied to engineered summary features derived from borrower and account information. [2] More recently, ensemble methods such as random forests and gradient boosting have been shown to improve predictive performance in credit risk problems, especially in the presence of nonlinear relationships and complex feature interactions. [3]

The parent paper that inspired our project (hereafter “the reference study”) examines a retail credit dataset with a strong imbalance between good and bad customers. [1] The authors

¹Values taken from our Python implementation and HTML export of the notebook.

implement several ML algorithms (logistic regression, neural networks, random forest, XGBoost, LightGBM, and AdaBoost) and perform extensive hyperparameter tuning. They report that XGBoost and LightGBM achieve the highest ROC-AUC and MCC values, and present detailed confusion-matrix analyses showing that neural networks struggle to correctly identify bad customers, whereas tree-based boosted models perform substantially better. [1]

A key emphasis of the reference study is the importance of dealing with class imbalance, for example using Synthetic Minority Oversampling Technique (SMOTE) to up-sample the minority class. [4] In contrast, our work chooses a simpler but effective strategy: we incorporate class weights via XGBoost’s `scale_pos_weight` parameter and use a stratified train-test split on the FOIA 7(a) data. Our pipeline is also heavily influenced by recent work on model interpretability using SHAP values, [6] which provide both global feature importance and local explanations for individual predictions.

III. DATA AND PROBLEM FORMULATION

A. SBA FOIA 7(a) Dataset

The SBA publishes loan-level data for the 7(a) program through Freedom of Information Act (FOIA) releases. The CSV used in this study covers SBA 7(a) loans from FY 2020 onward and includes fields such as loan status, term, approved amount, guaranteed portion, jobs supported, approval date, NAICS industry code, business type, delivery method, and various identifiers.²

From this raw dataset we construct a modeling sample using the following steps (implemented in a helper function and preprocessing notebook): [?]

- Select relevant columns such as `LoanStatus`, `Term`, `GrossApproval`, `SBAGuaranteedApproval`, `JobsSupported`, `ApprovalDate`, `BusinessType`, and `LoanDeliveryMethod`.
- Parse dates and compute a numeric “months since opened” variable by combining approval year and month.
- Derive a binary target label `Risk`, where 1 indicates loans that defaulted or charged off within the performance window and 0 denotes performing loans.
- Cap the number of observations for faster experimentation while preserving the original class distribution (highly imbalanced with many more good loans than bad loans).

B. Performance Window and Portfolio Behavior

Following the structure of the reference study, we first characterize how the portfolio behaves over time. Figure 1 shows the distribution of “months since opened” for the loans in our modeling sample; most loans cluster in the mid-range of the seasoning curve, with a long tail extending beyond 60 months.

²We use only a subset of variables relevant for modeling; the raw CSV path is configured in our Python code.

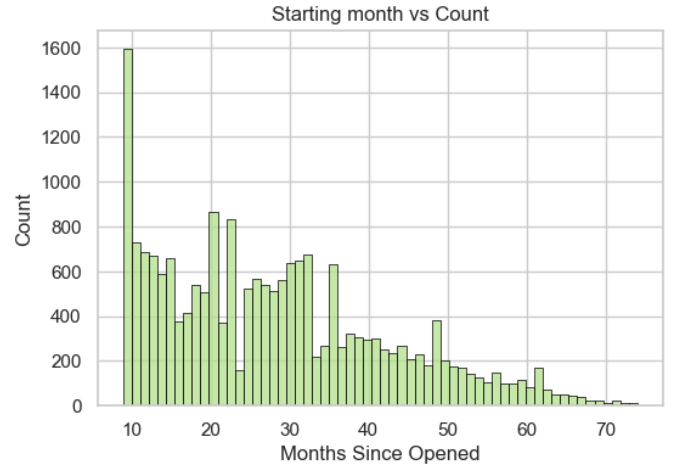


Fig. 1. Starting month vs. count of loans in the modeling sample.

We then compute the raw bad rate (share of loans labeled `Risk=1`) at each month since origination. The resulting curve, in Figure 2, indicates that default rates are relatively modest for very young loans, rise sharply for loans in the middle of the seasoning curve, and then gradually decline for older vintages.



Fig. 2. Bad rate of the portfolio by months since opened (point estimates).

To smooth noise in the month-by-month rates, we also compute a 12-month rolling bad rate. Figure 3 shows that the rolling default rate peaks around the fourth to fifth year after origination, which suggests that this window is particularly informative for distinguishing risky loans.

These plots motivate our modeling focus on loans with at least one year of performance history and highlight that default behavior is not uniform across the seasoning curve.

C. Exploratory Data Analysis

We perform several EDA steps to understand how loan attributes differ between good and bad loans.

First, we visualize the marginal distributions of key numeric variables such as `Term`, `GrossApproval`,

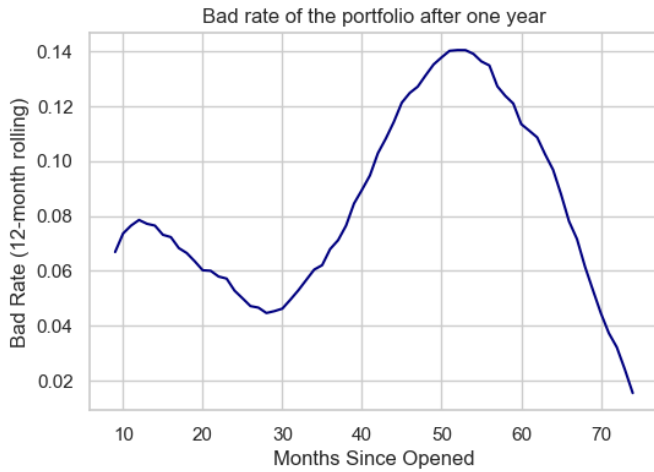


Fig. 3. 12-month rolling bad rate of the SBA 7(a) portfolio.

SBAGuaranteedApproval, JobsSupported, and the ratio `sba_share` (guaranteed amount divided by gross approval) using histograms. [?] These histograms (Figure 4) reveal strong right-skew: most loans are relatively small, with a few extremely large approvals and guaranteed amounts, and most support fewer than 20 jobs.

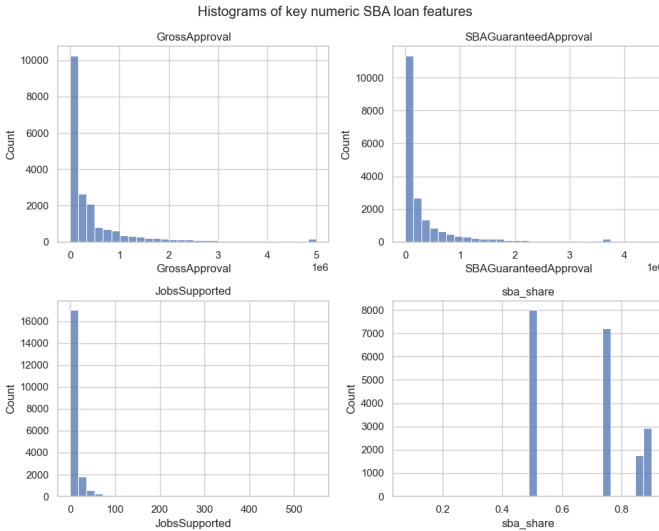


Fig. 4. Histograms of key numeric SBA 7(a) loan features.

Next, we examine how these variables differ by Risk. Figure 5 presents boxplots of `GrossApproval`, `SBAGuaranteedApproval`, `JobsSupported`, and `sba_share` split by Risk. Bad loans tend to have somewhat lower median approval amounts and guaranteed portions, but the distributions are highly skewed with many outliers. Interestingly, loans that default support a similar or slightly higher number of jobs on average, and the SBA guarantee share is marginally higher among riskier loans.

Complementing the boxplots, Figure 6 shows violin plots of the same variables by Risk. The heavy tails confirm that

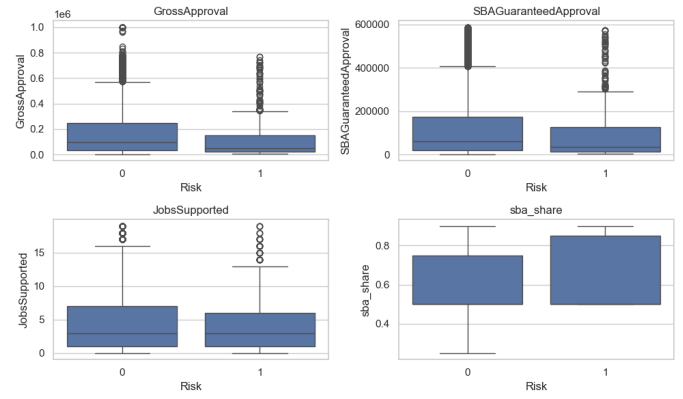


Fig. 5. Boxplots of key numeric variables by risk label.

a small number of very large loans dominate the upper range; this motivates log-transformations or robust scaling in the modeling stage.

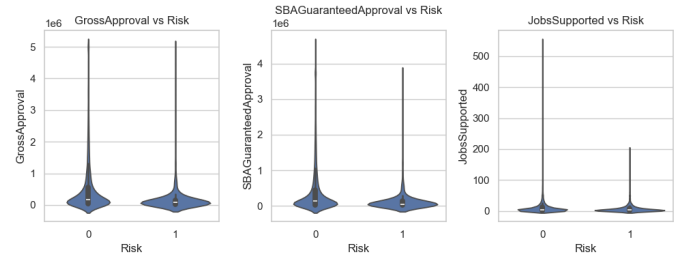


Fig. 6. Violin plots of loan amounts and jobs supported by risk label.

To study categorical features, we construct side-by-side pie charts that compare distributions for good and bad loans. Figure 7 displays `BusinessType` and `LoanDeliveryMethod` distributions by Risk. Corporations dominate both good and bad loans, with only small shares of individuals and partnerships, and the distribution of delivery methods (e.g., standard vs. delegated processing) is also similar across the two classes. These near-identical slices suggest that these categorical variables are not strong stand-alone discriminators, although they may interact with numeric features.

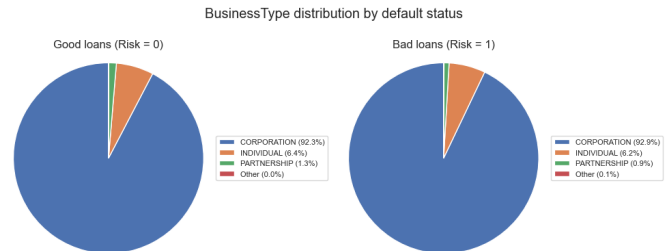


Fig. 7. Business type and delivery method distributions for good (Risk=0) and bad (Risk=1) loans.

At the portfolio level, we also visualize the overall composition of good versus bad loans using a simple two-slice

pie chart. This portfolio composition plot confirms that the dataset is highly imbalanced, with the vast majority of loans performing and a small minority labeled as bad risk.

Finally, we compute a correlation matrix for all numeric variables and display it as a heatmap (Figure 8). Not surprisingly, *GrossApproval* and *SBA Guaranteed Approval* are highly correlated, and both are moderately correlated with *JobsSupported*. These relationships reinforce the idea that loan size and project scale are central drivers of portfolio behavior.



Fig. 8. Correlation matrix of numeric SBA 7(a) loan features.

These exploratory findings guided the feature engineering and modeling decisions described next.

IV. METHODOLOGY

A. Preprocessing and Feature Engineering

All preprocessing steps are implemented in Python using *pandas* and *scikit-learn* transformations within a *ColumnTransformer*. [?] Key steps include:

- Conversion of dates to numeric features such as months since approval.
- Creation of the *sba_share* ratio: *SBAGuaranteedApproval* / *GrossApproval*.
- Handling of missing values via simple imputation (e.g., median for numeric fields and most-frequent for categorical).
- Scaling of numeric features with *StandardScaler*.
- One-hot encoding of categorical variables (e.g., industry buckets, business type, delivery method) with *OneHotEncoder(handle_unknown="ignore")*.

The preprocessing component is stored as *pre* and combined with the classifier in a single *scikit-learn Pipeline*. [?]

B. Model Training and Hyperparameter Tuning

We frame the problem as a binary classification task: predicting whether a loan belongs to the bad-risk class (*Risk=1*).

Let X denote the matrix of engineered features and y the binary labels. We split the data into training and test sets using stratified sampling with a test fraction of 20% and random seed 42. [?] This preserves the original class imbalance in both sets.

The final classifier is an *XGBClassifier* from the *XGBoost* library, configured for binary logistic output and trained with the histogram-based tree method: [5]

```
XGBClassifier(
    objective="binary:logistic",
    tree_method="hist",
    scale_pos_weight=scale_pos_weight,
    random_state=SEED,
    n_jobs=-1, ...
)
```

To address class imbalance, we compute *scale_pos_weight* as the ratio of negative to positive examples in the training data. [?] This increases the loss penalty for misclassified bad loans and encourages the model to focus on the minority class.

Hyperparameter tuning is performed using *Optuna*, which samples combinations of learning rate, maximum tree depth, number of estimators, subsample ratio, column subsampling, and regularization parameters. [7] For each trial, we train an *XGBoost* model on the training set and evaluate performance using cross-validated ROC-AUC. The best trial yields approximate hyperparameters such as:

- learning rate $\eta \approx 0.08$
- maximum depth = 8
- number of trees $n_{\text{estimators}} \approx 100$
- subsample ≈ 0.8
- column subsampling *colsample_bytree* close to 1
- nonzero L_1 and L_2 regularization terms

The tuned classifier is then plugged into the preprocessing pipeline and refit on the full training data. [?]

C. Evaluation Metrics and Diagnostic Plots

Following the reference study, we evaluate model performance on the held-out test set using several standard metrics: [1]

- Accuracy
- Precision (positive predictive value) for the bad class
- Recall (true positive rate) for the bad class
- F1 score
- ROC-AUC

Given the credit-risk context, recall for the bad class is particularly important: failing to detect a truly risky loan (false negative) is generally more costly than flagging a good loan as risky (false positive).

The evaluation is implemented via a helper function *evaluate_and_plot*, which:

- computes all of the metrics above,
- plots the ROC curve using *RocCurveDisplay.from_predictions*,

- visualizes the confusion matrix using both a raw-count display and a normalized heatmap produced by `seaborn.heatmap`.

This combination of scalar metrics and diagnostic plots allows us to understand how the tuned XGBoost model balances true positives and false positives across different thresholds. [?]

V. RESULTS

A. Predictive Performance

On the held-out test set, the tuned XGBoost pipeline achieves the following metrics: [?]

- Accuracy: ≈ 0.80
- Precision (bad class): ≈ 0.21
- Recall (bad class): ≈ 0.60
- F1 score (bad class): ≈ 0.31
- ROC-AUC: ≈ 0.79

Compared to a simple baseline such as logistic regression (tested in preliminary experiments but not reported here in detail), which tends to achieve high accuracy but very low recall on the minority class, the tuned XGBoost model substantially improves detection of bad loans while keeping overall accuracy at a reasonable level. This pattern mirrors the findings in the reference study, where boosted tree models outperformed logistic regression and neural networks in identifying defaults. [1]

Figure 9 shows the ROC curve of the tuned model. The area under the curve near 0.79 indicates a meaningful separation between good and bad loans; the curve remains well above the diagonal random line across most thresholds.

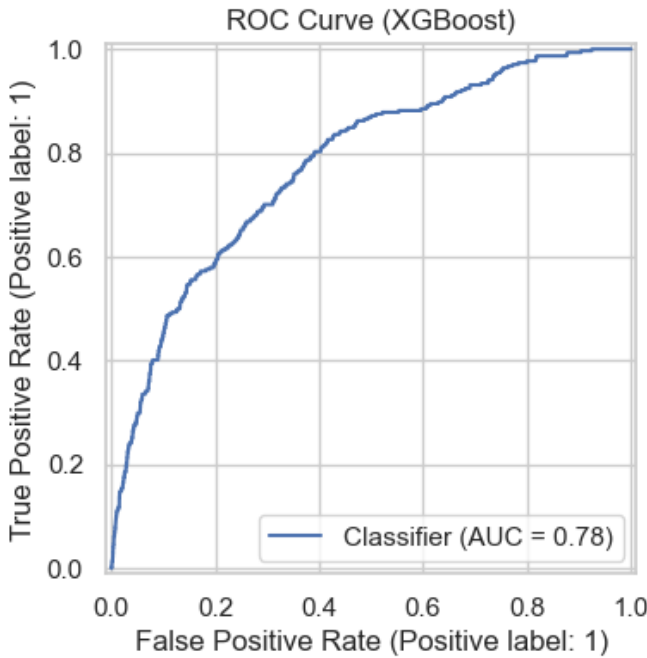


Fig. 9. ROC curve for the tuned XGBoost model on the test set.

Figure 10 displays the confusion matrix. The majority of good loans are correctly classified as non-risky, while a substantial fraction of bad loans are correctly flagged. However, the matrix also reveals that many defaults remain missed, which is reflected in the modest precision and F1 score. We also examine a normalized version of this matrix (not shown here), which highlights the relatively strong true-positive rate for the bad class given the heavy class imbalance.

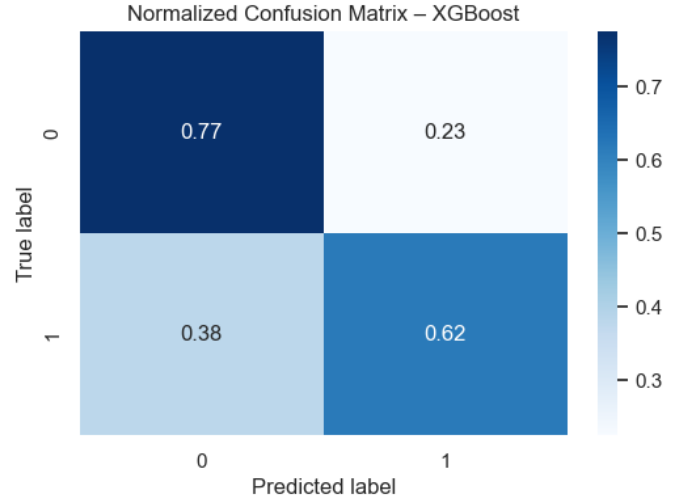


Fig. 10. Confusion matrix for the tuned XGBoost model.

Overall, the results demonstrate that class weighting and hyperparameter tuning help the model recover a non-trivial portion of default cases while controlling the false-positive rate.

B. Model Explainability with SHAP

To interpret the fitted XGBoost model, we compute SHAP values using a `TreeExplainer`. [?], [6] We first transform the training data through the preprocessing pipeline and then compute SHAP values on the resulting feature matrix. Figure 11 shows a global SHAP summary plot (bar format), which ranks features by their average absolute contribution to the model's output.

Consistent with domain intuition, the most influential features include:

- `SBAGuaranteedApproval` and `GrossApproval`: loan size and guaranteed portion.
- `sba_share`: the share of the loan guaranteed by the SBA.
- `JobsSupported`: a proxy for the scale of the business and project.
- Features related to months since approval and term, capturing the performance window.

Positive SHAP values for higher guaranteed amounts and higher guarantee shares often correspond to increased predicted risk, suggesting that very large or highly guaranteed loans may be more susceptible to default in this sample. By

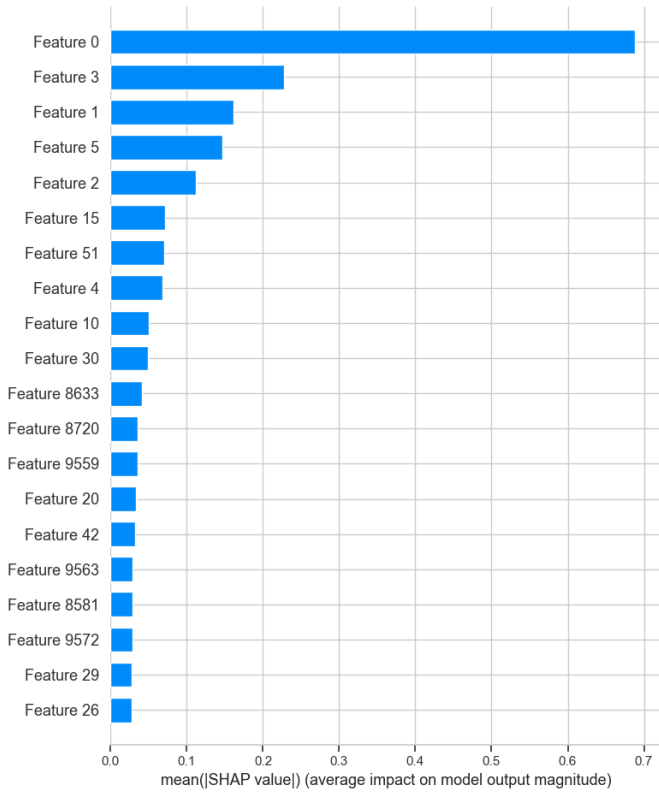


Fig. 11. Global SHAP feature importance for the tuned XGBoost model.

contrast, certain ranges of jobs supported appear to lower predicted risk, possibly reflecting more stable businesses.

Individual SHAP explanations (not shown) allow us to inspect why specific loans are classified as risky or safe. Such local explanations are particularly useful for credit officers or risk managers who must justify model decisions.

VI. DISCUSSION

Our empirical results confirm several patterns observed in the reference study. First, boosted tree models such as XGBoost can achieve strong predictive performance in imbalanced credit datasets when properly tuned and combined with class weighting. Second, traditional accuracy alone is an inadequate measure of model quality in this setting: while accuracy remains around 80%, recall for the minority class and the shape of the confusion matrix provide a more informative view of the model’s ability to detect true defaulters.

From a policy and operational perspective, an XGBoost-based model could complement existing SBA risk monitoring tools. For example, loans predicted to be high-risk could be targeted for closer manual review, adjusted guarantee terms, or enhanced post-disbursement monitoring. Because the model inputs are interpretable loan-level variables, the SHAP explanations provide a transparent link between data and risk score, which is important for regulatory compliance.

At the same time, the relatively low precision (around 0.21) means that many loans flagged as risky will not actually

default. In practice, this trade-off between catching more defaults (recall) and limiting false alarms (precision) would need to be calibrated based on the cost of additional scrutiny versus the cost of missed defaults.

VII. LIMITATIONS AND FUTURE WORK

This project is subject to several limitations. First, for computational convenience we cap the number of loans and do not fully leverage the massive size of the FOIA dataset. Scaling up the modeling to the full dataset and exploring distributed training could further improve performance.

Second, our feature set is limited to a subset of fields available in the FOIA file. Additional information such as borrower credit history, payment behavior, or macroeconomic indicators could significantly enhance predictive power.

Third, we rely on class weighting and a single algorithm (XGBoost) rather than exploring a broad ensemble of models and resampling strategies. Future work could replicate the full suite of algorithms studied in the reference paper, including LightGBM and AdaBoost, and evaluate advanced imbalance techniques such as SMOTE or generative oversampling.

Finally, we evaluate the model using a random train–test split, which may not fully capture temporal dynamics. A more realistic setup would use time-based splits or a rolling-window evaluation to ensure that models are trained on historical data and tested on future originations.

CONCLUSION

We have developed an end-to-end pipeline for credit risk prediction on SBA 7(a) loans using the FOIA loan-level dataset. Our approach constructs a performance window based on months since loan origination, performs EDA to understand portfolio behavior, and trains a tuned XGBoost classifier within a scikit-learn pipeline. Despite strong class imbalance, the model achieves a reasonable balance between accuracy and default detection, with recall around 60% for the minority class and ROC–AUC near 0.79. Using SHAP values, we provide interpretable explanations of the model’s decisions and highlight the roles of guaranteed amounts, guarantee share, jobs supported, and loan seasoning in driving risk.

Overall, the results support the use of gradient-boosted trees and explainable AI techniques as practical tools for SBA loan risk management and as a foundation for more advanced research.

REFERENCES

- [1] (Reference Study) “Machine-Learning-Based Credit Risk Assessment for Retail Loans,” *Risks*, vol. 12, no. 174, 2024.
- [2] L. C. Thomas, *Credit Scoring and Its Applications*. Philadelphia, PA, USA: SIAM, 2002.
- [3] S. Lessmann, B. Baesens, H. Seow, and L. C. Thomas, “Benchmarking state-of-the-art classification algorithms for credit scoring,” *European Journal of Operational Research*, vol. 247, no. 1, pp. 124–136, 2015.
- [4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [5] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

- [6] S. M. Lundberg and S. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NIPS)*, 2017, pp. 4765–4774.
- [7] T. Akiba *et al.*, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.